
aliases: - 11-closure-decorator

闭包 + 装饰器：搞懂 @xxx 到底在干什么

前置：已掌握函数定义、函数嵌套、`@property` 和 `@classmethod` 目标：理解装饰器原理，看懂 FastAPI 的 `@app.get("/path")` 用时：约 15 分钟

相关笔记：[函数基础](#) · [Python 基础补漏（作用域/*args）](#) · [课堂老师函数程序对照](#)

一、热身：函数也是变量

先接受一个事实：**Python 里函数和数字、字符串一样，可以赋值、可以当参数传。**

```
def say_hello(name):  
    return f"你好, {name}!"  
  
# 函数可以赋值给变量  
greeter = say_hello  
print(greeter("张三")) # 你好, 张三!  
  
# 函数可以当参数传  
def run(func, arg):  
    return func(arg)  
  
print(run(say_hello, "李四")) # 你好, 李四!
```

这不神奇——只是 Python 把函数当作"一等公民"（first-class citizen），和整数、字符串地位一样。

二、闭包：函数记住了它出生时的环境

一个现象

```
def make_multiplier(n):
    def multiplier(x):
        return x * n      # multiplier 用了外面的 n
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5))  # 10
print(triple(5))  # 15
```

`make_multiplier(2)` 结束之后，按理说变量 `n=2` 应该消失了——但 `double(5)` 仍然记得 `n=2`。

这就是闭包：内部函数 `multiplier` 把外部函数 `make_multiplier` 的变量 `n` 给"记住"了。

闭包的三个条件

1. 函数里定义函数（嵌套）
2. 内部函数用了外部函数的变量
3. 外部函数把内部函数返回出去

闭包的经典用途：计数器

```
def create_counter(start=0):
    count = start
    def counter():
        nonlocal count      # ← 声明 count 不是本地变量，是来自外层的
        count += 1
        return count - 1
    return counter

c = create_counter(10)
print(c())  # 10
print(c())  # 11
print(c())  # 12
```

`nonlocal` 的作用：告诉 Python "这个 `count` 不是本函数的局部变量，来自外层函数"。没有 `nonlocal`，内部函数里给 `count += 1` 会报错。

```
# 没有 nonlocal 的后果
def bad_counter(start=0):
    count = start
    def counter():
        count += 1    # ✗ UnboundLocalError
        return count
    return counter
```

为什么需要 `nonlocal`：Python 的变量赋值机制——`count += 1` 等价于 `count = count + 1`，这会在内部函数新建一个局部变量 `count`，和外层的 `count` 没关系。`nonlocal` 说“别新建，用外层的”。

一句话记

闭包 = 内部函数 + 它记住的外部变量。`nonlocal` 让内部函数能修改这些“记住的”变量。

三、装饰器：给函数套一层包装

装饰器是什么

装饰器 = 接收一个函数，返回一个新函数的函数。

```
def log_calls(func):
    """装饰器：在调用前后打日志"""
    def wrapper(*args, **kwargs):
        print(f"调用 {func.__name__} ()")
        result = func(*args, **kwargs)
        print(f"{func.__name__} () 返回 {result}")
        return result
    return wrapper
```

用法：

```
# 方式1：手动套
def add(a, b):
    return a + b
```

```

add = log_calls(add)      # add 被替换成了 wrapper
print(add(3, 5))
# 调用 add()
# add() 返回 8
# 8

# 方式2: @ 语法糖（完全等价）
@log_calls
def add(a, b):
    return a + b
# ↑ 这两行等价于 add = log_calls(add)

```

@ 只是一个快捷写法，它把下面的函数作为参数传给装饰器，然后用返回值替换原函数。

装饰器的数据流

```

@log_calls
def add(a, b):
    return a + b

```

等价于：

```

add = log_calls(add)
↓
log_calls 接收 add 函数
↓
返回 wrapper 函数
↓
add 被替换成 wrapper
↓
调用 add(3, 5) → 实际执行 wrapper(3, 5)
↓
wrapper 打日志 → 调原 add(3, 5) → 返回结果

```

你已经用过的装饰器

```

@property          # 把方法变成属性读取
@classmethod       # 第一个参数变成 cls 而不是 self

```

```
@staticmethod # 去掉 self 和 cls
```

为什么要用闭包来实现装饰器？

因为装饰器需要**记住原函数**（外部变量 `func`），并在 `wrapper` 里调用它——这天然就是个闭包。

```
def log_calls(func): # ← 外部函数，接收原函数
    def wrapper(*args, **kwargs): # ← 内部函数（包装）
        # func 被 wrapper 记住了（闭包）
        result = func(*args, **kwargs)
        return result
    return wrapper # ← 返回内部函数
```

破坏实验

```
# 如果装饰器什么都不做？
def do_nothing(func):
    return func # 直接返回原函数

@do_nothing
def hello():
    return "Hello!"

print(hello()) # ? 还能正常调吗？

# 如果装饰器返回的不是函数？
def return_int(func):
    return 42

@return_int
def hello():
    return "Hello!"

print(hello) # ? hello 变成什么了？
# hello() # ? 这个会怎样？
```

四、带参数的装饰器： `@app.get("/path")`

三层嵌套

FastAPI 里这种写法你见过：

```
app = FastAPI()

@app.get("/hello")
def hello():
    return {"msg": "Hello!"}
```

这里 `@app.get("/hello")` 不是普通的装饰器——它先调用了 `app.get("/hello")` 方法，这个方法的返回值才是真正的装饰器。

三层结构：

```
第一层：接收参数（"/hello"）
    ↓
第二层：接收函数（hello）
    ↓
第三层：包装函数（wrapper）
```

模拟实现

```
class FakeApp:
    def __init__(self):
        self.routes = {}          # 存路由表

    def get(self, path):          # ← 第一层：接收路径参数
        def decorator(func):     # ← 第二层：接收函数（真正的装饰器）
            self.routes[path] = func  # → 把路径和函数注册到路由表
            return func           # → 返回原函数（不包装，因为是注册模式）
        return decorator         # ← 返回装饰器
```

执行流程：

```
@app.get("/hello")
def hello():
    return "Hello!"
```

等价于：

```
@app.get("/hello")    →  app.get("/hello")    →  返回 decorator
@decorator             →  decorator(hello)     →  把 hello 注册到 self.routes["/hello"]
def hello():
    return "Hello!"
```

完整的例子

```
app = FakeApp()

@app.get("/hello")
def hello():
    return "Hello!"

@app.get("/status")
def status():
    return {"status": "ok"}

print(app.routes)
# {'/hello': <function hello>, '/status': <function status>}
```

对比普通装饰器：

	普通装饰器	@app.get("/path")
层数	2层（外=收函数，内=包装）	3层（外=收参数，中=收函数，内=包装）
典型用途	打日志、计时、权限校验	路由注册、配置绑定
调用方式	@decorator	@obj.method(arg)

```
# 三层装饰器少一层的后果
def bad_decorator(path):
    # 少写了中间那层
    pass # 根本没有返回装饰器

@bad_decorator("/test") # 会报什么错?
def test():
    return "test"
```

五、多个装饰器叠加

```
def bold(func):
    def wrapper(*args, **kwargs):
        return f"<b>{func(*args, **kwargs)}</b>"
    return wrapper

def italic(func):
    def wrapper(*args, **kwargs):
        return f"<i>{func(*args, **kwargs)}</i>"
    return wrapper

@bold
@italic
def greet(name):
    return f"Hello, {name}!"

print(greet("World"))
```

输出什么？

执行顺序是**从下往上**装饰，**从上往下**执行：

```
原始 greet = "Hello, World!"

@italic 装饰后 → "<i>Hello, World!</i>"
↑ 先装饰 italic
```



```
@bold 装饰后 → "<b><i>Hello, World!</i></b>"
↑ 再装饰 bold
```

等价于：

```
greet = bold(italic(greet))
```

装饰器叠加顺序：离函数最近的最先装饰，离函数最远的最后包装。

总结

概念	一句话	什么时候用
函数即变量	函数可以赋值、当参数传	理解装饰器的基础
闭包	内部函数记住外部变量	计数器、缓存、装饰器底层实现
装饰器（2层）	<code>@xxx</code> = 接收函数，返回新函数	打日志、计时、权限校验
装饰器（3层）	<code>@xxx(arg)</code> = 先调外层收参数，返回装饰器	FastAPI 路由注册、配置绑定
叠加	<code>@A @B</code> = <code>A(B(func))</code>	多个独立关注点组合